

LAPORAN MILESTONE 3 TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA
Semantic Analysis



Disusun oleh:

Juan Oloando Simanungkalit 13524032

Edward David Rumahorbo 13524036

Reinhard Alfonzo Hutabarat 13524056

Manuel Timothy Silalahi 13524102

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL.GANESA 10, BANDUNG 40132
2025

DAFTAR ISI

DAFTAR ISI.....	1
BAB 1	
LANDASAN TEORI.....	1
1. Semantic Analysis.....	1
2. Type Checking.....	1
3. Symbol Table Management.....	2
4. Scope Resolution.....	2
5. Control Flow Validation.....	3
6. Abstract Syntax Tree.....	4
7. Decorated Abstract Syntax Tree.....	4
8. Attributed Grammar.....	4
9. Visitor / Visit Functions.....	5
BAB 2	
PERANCANGAN DAN IMPLEMENTASI.....	6
1. Perancangan.....	6
1.1. Arsitektur Umum.....	6
1.2. Struktur Program.....	6
2. Implementasi.....	8
2.1. Kakas yang Digunakan.....	8
2.2. Struktur Data ASTNode.....	8
2.3. Modul TypeSystem.....	9
2.4. Kelas SymbolTable.....	12
2.5. Kelas ASTBuilder.....	16
2.6. Kelas SemanticAnalyzer.....	20
2.7. Namespace SemanticErrors.....	29
2.8. Kelas ASTPrinter.....	30
2.9. Main.cpp.....	31
BAB 3	
PENGUJIAN.....	33
1. Test Case 1.....	33
2. Test Case 2.....	34
3. Test Case 3.....	38
4. Test Case 4.....	40
5. Test Case 5.....	42
BAB 4	
KESIMPULAN DAN SARAN.....	46
1. Kesimpulan.....	46

2. Saran.....	46
LAMPIRAN.....	48
REFERENSI.....	48

BAB 1

LANDASAN TEORI

1. Semantic Analysis

Semantic analysis merupakan tahap ketiga dalam pipeline compiler, dilaksanakan setelah *lexical analysis* dan *syntax analysis* selesai dikerjakan. Jika *syntax analysis* hanya memastikan bahwa urutan token membentuk struktur yang sesuai dengan *grammar*, maka *semantic analysis* bertugas memeriksa apakah program tersebut bermakna secara semantic, artinya apakah setiap konstruksi bahasa digunakan sesuai dengan aturan logika yang berlaku dalam bahasa pemrograman tersebut.

Dalam konteks kompilasi bahasa Arion, *semantic analyzer* menerima masukan berupa *Abstract Syntax Tree* (AST) yang dihasilkan oleh *parser* pada Milestone 2, kemudian menelusurinya untuk memeriksa sejumlah aturan semantic. Beberapa tanggung jawab utama *semantic analysis* meliputi pemeriksaan tipe data (*type checking*), resolusi scope (*scope resolution*), pengelolaan tabel simbol (*symbol table management*), serta validasi alur kontrol (*control flow validation*). Apabila semua pemeriksaan berhasil dilalui, *semantic analyzer* akan menghasilkan *Decorated Abstract Syntax Tree* (DAST) yang siap digunakan pada tahap selanjutnya.

2. Type Checking

Type checking adalah proses pemeriksaan kesesuaian tipe data pada setiap operasi, ekspresi, dan penugasan dalam program. Terdapat dua pendekatan utama dalam *type checking*: *static type checking* yang dilakukan pada waktu kompilasi sebelum program dijalankan, dan *dynamic type checking* yang dilakukan pada saat program sedang berjalan. Bahasa Arion, seperti halnya Standard Pascal, menggunakan *static type checking* sehingga seluruh kesalahan tipe terdeteksi pada fase kompilasi.

Dalam implementasi Milestone 3, pemeriksaan tipe dikelola oleh modul *TypeSystem* yang menyediakan fungsi-fungsi inferensi dan kompatibilitas tipe. Terdapat tiga mekanisme utama yang diimplementasikan:

- **Inferensi tipe biner (*inferBinary*):** menghitung tipe hasil dari operasi biner. Aturan yang berlaku antara lain: *Integer* op *Integer* menghasilkan *Integer*, pembagian "/" selalu menghasilkan *Real*, operator *div* dan *mod* hanya berlaku untuk *Integer*, serta operator relasional selalu menghasilkan *Boolean*.
- **Inferensi tipe unary (*inferUnary*):** menghitung tipe hasil dari operasi unary. Operator *not* hanya berlaku untuk *Boolean*, sedangkan unary "-" dan "+" hanya berlaku untuk tipe numerik.

- **Kompatibilitas tipe** (*isCompatible* dan *isAssignmentCompatible*): memeriksa apakah dua tipe dapat dioperasikan bersama atau saling diisikan. Contohnya, *Real* := *Integer* valid karena terjadi pelebaran tipe (*widening*), namun *Integer* := *Real* tidak valid karena berpotensi kehilangan informasi.

3. *Symbol Table Management*

Symbol table adalah struktur data sentral dalam compiler yang menyimpan informasi tentang seluruh identifier yang dideklarasikan dalam program, mencakup nama, tipe, kelas objek, level leksikal, dan alamat memori. Informasi ini dibutuhkan oleh *semantic analyzer* untuk melakukan pemeriksaan tipe, resolusi nama, dan alokasi memori.

Dalam implementasi Milestone 3, tabel simbol dikelola oleh kelas *SymbolTable* yang menggunakan tiga tabel terpisah mengikuti konvensi model Arion:

- **tab**: tabel utama yang menyimpan setiap identifier yang dideklarasikan. Setiap entri *TabEntry* memiliki atribut *identifier*, *link* (penunjuk ke identifier sebelumnya dalam scope yang sama), *obj* (kelas objek seperti *Variable*, *Constant*, *Procedure*, dst.), *type*, *ref*, *nrm* (1 = by-value, 0 = by-reference), *lev* (level leksikal), dan *adr* (offset pada stack frame). Identifier pengguna dimulai dari indeks 33, sementara indeks 0–32 dicadangkan sebagai *reserved words*.
- **btabs**: tabel blok yang menyimpan informasi tentang setiap blok program seperti prosedur, fungsi, dan record. Setiap entri *BTabEntry* memiliki atribut *last* (indeks identifier terakhir dalam blok), *lpar* (indeks parameter terakhir), *psze* (total ukuran parameter), dan *vsze* (total ukuran variabel lokal).
- **atab**: tabel array yang menyimpan informasi tentang tipe array. Setiap entri *ATabEntry* memiliki atribut *xtyp* (tipe indeks, harus ordinal), *etyp* (tipe elemen), *low*, *high* (batas indeks), *elsz* (ukuran elemen), dan *size* (total ukuran array).

Tabel simbol juga dilengkapi dengan identifier predefined yang diinisialisasi sebelum analisis dimulai, mencakup tipe dasar (*Integer*, *Real*, *Char*, *Boolean*, *String*), konstanta bawaan (*True*, *False*), serta prosedur input/output bawaan (*writeln*, *readln*, *write*, *read*).

4. *Scope Resolution*

Scope atau lingkup adalah wilayah dalam teks program di mana suatu identifier berlaku dan dapat dirujuk. Bahasa Arion menggunakan *lexical scoping* (atau *static scoping*), di mana lingkup sebuah identifier ditentukan berdasarkan struktur teks program secara statis, bukan berdasarkan alur eksekusi saat runtime.

Scope resolution adalah proses menentukan identifier mana yang dirujuk ketika suatu nama digunakan dalam program. Dalam implementasi *SymbolTable*, resolusi scope dilakukan dengan dua fungsi:

- **lookup(name)**: mencari identifier dari scope terdalam menuju scope global. Pencarian dilakukan secara linear mundur dari akhir tabel, melewati entry yang levelnya melebihi level saat ini (artinya sudah keluar dari scope tersebut). Pencarian bersifat *case-insensitive* sesuai konvensi Pascal.
- **lookupCurrentScope(name)**: mencari identifier hanya dalam scope aktif saat ini. Fungsi ini digunakan oleh *declare()* untuk mendeteksi redeclaration dalam scope yang sama.

Manajemen scope diimplementasikan menggunakan pasangan fungsi *enterBlock()* dan *leaveBlock()*. Ketika *enterBlock()* dipanggil, sebuah entry baru ditambahkan ke *btabs* dan level leksikal dinaikkan. Saat *leaveBlock()* dipanggil, ukuran variabel lokal (*vsz*) disimpan ke entri *btabs* yang bersangkutan, dan level leksikal diturunkan. Setelah *leaveBlock()*, semua identifier pada level yang lebih tinggi dari level saat ini tidak akan dikembalikan oleh *lookup()*, sehingga variabel lokal tidak bocor ke scope luar.

5. Control Flow Validation

Control flow validation adalah pemeriksaan semantic yang memastikan bahwa alur kendali program valid secara logis, meskipun secara sintaksis sudah benar. Tanpa pemeriksaan ini, program yang secara struktur valid namun berperilaku tidak terdefinisi masih bisa lolos tahap parsing. Dalam konteks bahasa Arion, validasi alur kontrol yang perlu diperiksa antara lain:

- **Kondisi boolean pada pernyataan kontrol**: ekspresi kondisi pada *if*, *while*, dan *repeat-until* harus bertipe *Boolean*. Menggunakan ekspresi bertipe *Integer* sebagai kondisi adalah kesalahan semantic.
- **Variabel kontrol pada pernyataan for**: variabel yang digunakan sebagai pencacah harus bertipe ordinal (*Integer*, *Char*, atau *Boolean*). Ekspresi batas bawah dan batas atas juga harus kompatibel dengan tipe variabel kontrol tersebut.
- **Nilai pada pernyataan case**: ekspresi selektor harus bertipe ordinal, dan setiap nilai *case label* harus kompatibel dengan tipe selektor.
- **Nilai kembalian fungsi**: fungsi harus memiliki jalur yang mengisikan nilai ke identifier fungsi itu sendiri (nama fungsi berlaku sebagai variabel hasil) agar nilai kembalian terdefinisi.

6. *Abstract Syntax Tree*

Abstract Syntax Tree (AST) adalah representasi pohon dari struktur logis sebuah program. Berbeda dengan *Parse Tree* yang dihasilkan pada Milestone 2 dan mempertahankan seluruh simbol grammar, AST hanya berisi informasi yang esensial untuk analisis semantic dan tahap compile berikutnya. Simbol-simbol struktural seperti tanda kurung, titik koma, dan kata kunci pembatas dibuang, sehingga pohon menjadi lebih ringkas namun tetap mempertahankan hubungan hierarkis antar konstruksi program.

Dalam implementasi Arion Compiler, setiap node AST direpresentasikan oleh kelas *ASTNode* yang memiliki atribut *type* (jenis node seperti *ProgramNode*, *AssignNode*, *BinaryOpNode*, dst.), *value* (nilai literal atau nama identifier), dan *children* (daftar node anak). AST dibangun oleh *ASTBuilder* pada fase sebelumnya dan menjadi masukan utama bagi *SemanticAnalyzer* pada Milestone 3 ini.

7. *Decorated Abstract Syntax Tree*

Decorated Abstract Syntax Tree (DAST) adalah AST yang ditambahkan dengan informasi semantic sebagai hasil dari proses *semantic analysis*. Setiap node yang relevan pada pohon diberikan anotasi berupa atribut tambahan yang dibutuhkan untuk fase pembuatan kode atau interpretasi. Proses pemberian anotasi ini dikenal sebagai *attributed translation* atau *decoration*. Dalam konteks Arion Compiler, dekorasi yang ditambahkan pada setiap node antara lain:

- **Informasi tipe (*SemanticType*):** setiap node ekspresi dianotasi dengan tipe hasilnya. Contohnya, node *BinaryOpNode* untuk operasi " $a + b$ " akan dianotasi dengan tipe *Integer* jika a dan b keduanya bertipe *Integer*, atau *Real* jika salah satunya bertipe *Real*.
- **Referensi ke tabel simbol (*tab*, *btabs*, *atab*):** node yang merujuk identifier dianotasi dengan indeks entri pada *tab*. Node yang merepresentasikan prosedur atau fungsi merujuk ke entri *btabs*. Node yang merepresentasikan variabel array merujuk ke entri *atab* yang menyimpan informasi dimensi dan tipe elemen.

DAST yang dihasilkan pada akhir *semantic analysis* menjadi representasi lengkap program yang siap diinterpretasikan atau dikompilasi lebih lanjut, karena seluruh informasi tipe dan referensi simbol sudah tersedia langsung pada setiap node tanpa perlu pencarian ulang.

8. *Attributed Grammar*

Attributed grammar adalah perluasan dari *Context-Free Grammar* (CFG) yang melengkapi setiap simbol grammar dengan atribut-atribut yang menyimpan informasi semantic. Setiap aturan produksi dalam grammar diperluas dengan *semantic action*, yaitu

aturan komputasi yang menentukan bagaimana nilai atribut setiap simbol diturunkan atau diwarisi dari simbol-simbol lain dalam produksi tersebut. Terdapat dua kategori atribut dalam *attributed grammar*:

- **Synthesized attributes**: nilai atribut sebuah simbol dihitung dari atribut simbol-simbol anaknya dalam pohon derivasi. Contohnya, tipe hasil dari ekspresi $a + b$ (disintesis dari tipe a dan tipe b) adalah *synthesized attribute*. Ini adalah jenis atribut yang paling umum digunakan dalam *type checking*.
- **Inherited attributes**: nilai atribut sebuah simbol diwariskan dari simbol induk atau simbol saudaranya. Contohnya, informasi scope atau level leksikal yang diteruskan dari blok luar ke blok dalam adalah *inherited attribute*.

Pada implementasi Milestone 3, konsep *attributed grammar* direalisasikan melalui fungsi *visit* pada *SemanticAnalyzer* yang menelusuri AST, menghitung atribut setiap node berdasarkan atribut anak-anaknya, dan menyimpan hasilnya sebagai dekorasi pada DAST

9. Visitor / Visit Functions

Visitor pattern adalah pola desain yang memisahkan operasi yang dilakukan pada sebuah struktur data dari struktur data itu sendiri. Dalam konteks kompilasi, pola ini digunakan untuk menelusuri AST sambil menerapkan berbagai pemeriksaan semantic tanpa mengubah definisi kelas node AST.

Dalam implementasi *SemanticAnalyzer* pada Milestone 3, fungsi *visit* diimplementasikan dengan pendekatan *recursive tree traversal*. Setiap jenis node AST memiliki penanganan tersendiri dalam fungsi *visit* yang melakukan urutan pemeriksaan berikut:

- Menelusuri seluruh node anak secara rekursif (*bottom-up traversal*), sehingga informasi tipe dari sub-ekspresi sudah tersedia sebelum ekspresi induk diperiksa.
- Membaca informasi tipe node anak dari hasil kunjungan sebelumnya dan menerapkan aturan inferensi tipe yang sesuai (memanggil *inferBinary*, *inferUnary*, *isCompatible*, atau *isAssignmentCompatible* dari modul *TypeSystem*).
- Mencari identifier yang dirujuk pada *SymbolTable* menggunakan *lookup()* dan melaporkan error jika identifier tidak ditemukan (*undeclared identifier*).
- Menyimpan hasil anotasi tipe dan referensi tabel simbol ke node AST yang bersangkutan, sehingga pohon yang sudah ditelusuri menjadi DAST.

Keunggulan pendekatan *visitor* adalah modularitas: setiap kategori pemeriksaan semantic dapat ditambahkan sebagai *visit function* baru tanpa mengubah struktur AST yang sudah ada. Ini mempermudah pengerjaan implementasinya, dengan sistem pengerjaan paralel.

BAB 2

PERANCANGAN DAN IMPLEMENTASI

1. Perancangan

1.1. Arsitektur Umum

Pada Milestone 3 Arion Compiler, program diperluas dengan menambahkan dua tahap baru ke dalam *pipeline* yang sudah ada: pembangunan *Abstract Syntax Tree* (AST) dari *Parse Tree* Milestone 2, serta analisis semantic (*Semantic Analysis*) terhadap AST yang dihasilkan. Secara umum, alur kerja program menjadi: Lexer menghasilkan token, Parser membangun *Parse Tree*, ASTBuilder mengkonversi *Parse Tree* menjadi AST yang lebih ringkas, SemanticAnalyzer menelusuri AST sambil mengisi tabel simbol dan mendekorasi setiap node dengan informasi tipe (*Decorated AST*), lalu ASTPrinter mencetak hasil akhir ke file output bersama isi tabel simbol.

Pipeline tiga tahap ini memisahkan tugas per tahap secara jelas: Parser hanya memeriksa *syntax*, ASTBuilder membuang informasi struktural yang tidak sesuai secara semantic, dan SemanticAnalyzer berkonsentrasi penuh pada pemeriksaan tipe dan validasi nama tanpa perlu menangani detail *grammar*. Pendekatan ini memudahkan pengujian setiap komponen secara independen.

1.2. Struktur Program

Program dikembangkan menggunakan bahasa C++ dan menambahkan beberapa file baru ke dalam folder `src/` yang sudah ada dari Milestone 2. Berikut adalah daftar file yang ditambahkan atau dimodifikasi beserta tugasnya:

Tabel 1 Struktur Program

File	Tanggung Jawab
AST.hpp	Definisi enum <code>ASTKind</code> , struct <code>SemanticType</code> (<code>TypeKind</code> , <code>ref</code> , <code>low</code> , <code>high</code> , <code>hasBounds</code>), dan kelas <code>ASTNode</code> yang menjadi node pada AST beserta atribut dekorasi <code>inferredType</code> , <code>tabIndex</code> , <code>blockIndex</code> , dan <code>lexicalLevel</code> .
ASTBuilder.hpp / ASTBuilder.cpp	Kelas <code>ASTBuilder</code> yang mengonversi <i>Parse Tree</i> dari Milestone 2 menjadi AST. Menyediakan fungsi <code>build()</code> sebagai titik masuk, serta

	fungsi-fungsi buildProgram(), buildDeclarationPart(), buildVarDeclaration(), buildSubprogramDeclaration(), buildExpression(), dan lain sebagainya.
ASTPrinter.hpp / ASTPrinter.cpp	Kelas ASTPrinter yang mencetak DAST ke output stream dengan format visual hierarkis. Setiap node dicetak beserta atribut dekorasinya: type, tab, block, dan level.
TypeSystem.hpp / TypeSystem.cpp	Modul sistem tipe. Mendefinisikan enum TypeKind dan helper konstruktor makeType(), makeIntType(), dst. Mengimplementasikan fungsi isCompatible(), isAssignmentCompatible(), inferBinary(), dan inferUnary() yang digunakan oleh SemanticAnalyzer untuk pemeriksaan dan inferensi tipe.
SymbolTable.hpp / SymbolTable.cpp	Kelas SymbolTable yang mengelola tiga tabel simbol: tab (identifier), btab (blok prosedur/fungsi/record), dan atab (tipe array). Menyediakan fungsi declare(), lookup(), lookupCurrentScope(), lookupInBlock(), enterBlock(), leaveBlock(), serta printTables() untuk mencetak ketiga tabel ke output.
SemanticErrors.hpp / SemanticErrors.cpp	Namespace SemanticErrors yang menyediakan fungsi-fungsi pembuat pesan kesalahan semantic yang terstandarisasi, seperti undeclaredIdentifier(), typeMismatchAssignment(), operatorMismatch(), conditionNotBoolean(), dan lain sebagainya.
SemanticAnalyzer.hpp / SemanticAnalyzer.cpp	Kelas SemanticAnalyzer yang menjadi inti analisis semantic.

	Menelusuri AST menggunakan fungsi <code>visit()</code> yang mendispatch ke handler spesifik per jenis node, mengisi <code>SymbolTable</code> , mendekorasi setiap node AST, dan mengumpulkan error semantic.
Main.cpp	Entry point program yang diperbarui. Mengintegrasikan seluruh komponen: Lexer, Parser, ASTBuilder, SemanticAnalyzer, ASTPrinter, dan <code>SymbolTable::printTables()</code> untuk menghasilkan output lengkap ke file.

2. Implementasi

2.1. Kakas yang Digunakan

Implementasi program menggunakan kakas yang sama seperti pada Milestone 2:

- Bahasa Pemrograman: C++, alasan pemilihan adalah karena kemampuan manajemen memori manual yang baik untuk membangun struktur pohon (*tree*), serta adanya *standar library* yang memadai seperti `std::vector`, `std::string`, dan lain sebagainya.
- Compiler: g++ (GNU Compiler Collection) yang mendukung standar C++17.

2.2. Struktur Data ASTNode

Setiap node pada AST direpresentasikan oleh struct *ASTNode* yang didefinisikan pada *AST.hpp*. Struct ini memiliki atribut inti untuk membangun pohon sekaligus atribut dekorasi yang diisi oleh SemanticAnalyzer:

```
struct ASTNode {
    ASTKind kind;
    std::string value;
    std::vector<ASTNode*> children;

    SemanticType inferredType;
    int tabIndex = -1;
    int blockIndex = -1;
    int lexicalLevel = 0;

    ASTNode(ASTKind kind, const std::string& value =
```

```

"" : kind(kind), value(value) {}

~ASTNode() {
    for (auto child : children) delete child;
}

void addChild(ASTNode* child) {
    if (child) children.push_back(child);
}
};

```

Atribut *kind* menyimpan jenis node menggunakan enum *ASTKind* (seperti *Program*, *Assign*, *BinOp*, *Var*, dst.), sedangkan *value* menyimpan nama identifier atau nilai literal. Atribut *inferredType* diisi oleh *SemanticAnalyzer* dengan tipe hasil inferensi, *tabIndex* menyimpan indeks entri pada *tab* untuk identifier yang sudah dideklarasikan, *blockIndex* menyimpan indeks blok pada *btabs*, dan *lexicalLevel* menyimpan level leksikal saat node dibuat. Destruktor kelas ini secara rekursif menghapus seluruh anak sehingga cukup memanggil *delete* pada node root untuk membebaskan seluruh memori pohon.

2.3. Modul *TypeSystem*

Modul *TypeSystem* mendefinisikan enum *TypeKind* dan menyediakan fungsi-fungsi untuk inferensi tipe dan pemeriksaan kompatibilitas. Modul ini dirancang agar dapat digunakan secara langsung oleh *SemanticAnalyzer* tanpa ketergantungan pada struktur pohon.

Fungsi *isCompatible()* dan *isAssignmentCompatible()*

```

bool isCompatible(const SemanticType& left, const
SemanticType& right) {

    if (left.kind == TypeKind::Unknown || right.kind
= TypeKind::Unknown)
        return false;

    if (isNumericType(left) && isNumericType(right))

```

```

        return true;

        if (left.kind == TypeKind::Subrange && right.kind
== TypeKind::Integer) return true;
        if (left.kind == TypeKind::Integer && right.kind
== TypeKind::Subrange) return true;

        if (left.kind == TypeKind::Subrange && right.kind
== TypeKind::Subrange) return true;

        return left.kind == right.kind && left.ref ==
right.ref;
    }

bool isAssignmentCompatible(const SemanticType&
target, const SemanticType& value) {
    if (target.kind == TypeKind::Unknown ||
value.kind == TypeKind::Unknown) return false;

    if (target.kind == TypeKind::Real && value.kind
== TypeKind::Integer) return true;

    if (target.kind == TypeKind::Subrange &&
value.kind == TypeKind::Integer) return true;
    if (target.kind == TypeKind::Integer &&
value.kind == TypeKind::Subrange) return true;
    if (target.kind == TypeKind::Subrange &&
value.kind == TypeKind::Subrange) return true;

    return target.kind == value.kind && target.ref ==
value.ref;
}

```

Fungsi *isCompatible()* memeriksa apakah dua tipe dapat dioperasikan bersama. *Integer* dan *Real* selalu kompatibel untuk operasi numerik, sedangkan *Subrange* kompatibel dengan *Integer* dan sesama *Subrange*. Untuk tipe lain seperti *Array* dan *Record*, field *ref* digunakan untuk memastikan hanya tipe yang sama persis yang dianggap kompatibel. Fungsi *isAssignmentCompatible()* menambah aturan

khusus bahwa *Real* := *Integer* valid (*widening*), namun *Integer* := *Real* tidak valid karena berpotensi kehilangan informasi desimal.

Fungsi inferBinary()

```
SemanticType inferBinary(const std::string& op, const SemanticType& left,
const SemanticType& right) {

    // jika salahs atu variable unknown, makas hasilnya jg unknown
    if (left.kind == TypeKind::Unknown || right.kind ==
TypeKind::Unknown)return makeUnknownType();

    // artimatika
    if (op == "+") {
        if (left.kind == TypeKind::String && right.kind == TypeKind::String)
            return makeStringType();
        if (left.kind == TypeKind::Integer && right.kind == TypeKind::Integer)
            return makeIntType();
        if (isNumericType(left) && isNumericType(right))
            return makeRealType();
        return makeUnknownType();
    }

    if (op == "-" || op == "*") {
        if (left.kind == TypeKind::Integer && right.kind == TypeKind::Integer)
            return makeIntType();
        if (isNumericType(left) && isNumericType(right))
            return makeRealType();
        return makeUnknownType();
    }

    if (op == "/") {
        if (isNumericType(left) && isNumericType(right))
            return makeRealType();
        return makeUnknownType();
    }

    if (op == "div") {
        if (left.kind == TypeKind::Integer && right.kind == TypeKind::Integer)
            return makeIntType();
        return makeUnknownType();
    }
}
```

```

if (op == "mod") {
    if (left.kind == TypeKind::Integer && right.kind == TypeKind::Integer)
        return makeIntType();
    return makeUnknownType();
}

// logika
if (op == "and" || op == "or") {
    if (left.kind == TypeKind::Boolean && right.kind == TypeKind::Boolean)
        return makeBoolType();
    return makeUnknownType();
}

if (isRelationalOp(op)) {
    if (isCompatible(left, right)) return makeBoolType();
    return makeUnknownType();
}

return makeUnknownType();
}

```

Fungsi *inferBinary()* menghitung tipe hasil operasi biner berdasarkan operator dan tipe kedua operandnya. Jika salah satu operand bertipe *Unknown*, hasil selalu *Unknown* untuk menghindari pelaporan error ganda. Operator "/" selalu menghasilkan *Real* sesuai standar Pascal, sedangkan *div* dan *mod* hanya berlaku untuk *Integer*. Seluruh operator relasional menghasilkan *Boolean* selama kedua operand kompatibel, sehingga ekspresi seperti $a < b$ menghasilkan tipe *Boolean* yang dapat langsung digunakan sebagai kondisi *if* atau *while*.

Type Compability Rule

Aturan kompatibilitas tipe yang digunakan dalam implementasi ini dibagi menjadi dua jenis: kompatibilitas untuk operasi (digunakan oleh *isCompatible()* pada pemeriksaan ekspresi biner dan akses array) dan kompatibilitas untuk assignment (digunakan oleh *isAssignmentCompatible()* pada pemeriksaan pernyataan penugasan). Berikut tabel lengkap aturan yang berlaku:

Tabel 2 Type Compatibility Rule Operasi(isCompatible)

Tipe Kiri	Tipe Kanan	Kompatibel
Integer	Integer	Ya
Integer	Real	Ya (numerik)
Real	Integer	Ya (numerik)
Real	Real	Ya (numerik)
Subrange	Integer	Ya ($\text{subrange} \subseteq \text{Integer}$)
Integer	Subrange	Ya ($\text{subrange} \subseteq \text{Integer}$)
Subrange	Subrange	Ya (keduanya subrange Integer)
T	T	Ya (kind sama dan ref sama)
Unknown	apapun	Tidak (error sudah dilaporkan)
apapun	Unknown	Tidak (error sudah dilaporkan)
T1	T2 (beda)	Tidak

Tabel 3 Type Compatibility Rule Assignment(isAssignmentCompatible)

Tipe Target	Tipe Nilai	Kompatibel
Real	Integer	Ya (widening)
Integer	Real	Tidak (narrowing, berpotensi kehilangan desimal)
Integer	Integer	Ya
Subrange	Integer	Ya
Integer	Subrange	Ya
Subrange	Subrange	Ya
Boolean	Boolean	Ya
Char	Char	Ya
String	String	Ya (ref harus sama)

Array(T)	Array(T)	Ya (ref ke atab harus sama)
Record(T)	Record(T)	Ya (ref ke btab harus sama)
Unknown	apapun	Tidak
T1	T2 (beda)	Tidak

Tabel 4 Aturan Inferensi Tipe Operasi Biner (inferBinary)

Operator	Tipe Kiri	Tipe Kanan	Tipe Hasil
+, -, *	Integer	Integer	Integer
+, -, *	Real	Real	Real
+, -, *	Integer	Real	Real
+, -, *	Real	Integer	Real
/	Integer	Integer	Real (selalu Real)
/	Real	Real	Real
div, mod	Integer	Integer	Integer
div, mod	Real	apapun	Unknown (error)
and, or	Boolean	Boolean	Boolean
and, or	non-Bool	apapun	Unknown (error)
=, <, <=, >, >=	T1 kompatibel T2	—	Boolean
=, <, <=, >, >=	T1 tidak kompatibel T2	—	Unknown (error)
+	String	String	String

Tabel 5 Aturan Inferensi Tipe Operasi Unary (inferUnary)

Operator	Tipe Operand	Tipe Hasil
not	Boolean	Boolean
not	non-Boolean	Unknown (error)
-, +	Integer	Integer

-, +	Real	Real
-, +	non-numerik	Unknown (error)

2.4. Kelas SymbolTable

File ini mengimplementasikan fungsi-fungsi yang berhubungan dengan struktur inti program Arion.

Fungsi declare() dan linked list per blok

```
int SymbolTable::declare(const std::string& name,
ObjectKind obj, const SemanticType& type, int nrm,
int adrOverride) {
    if (lookupCurrentScope(name)  $\neq$  -1)
        return -1;
    TabEntry e;
    e.identifier = name;
    e.obj = obj;
    e.type = type;
    e.nrm = nrm;
    e.lev = currentLevel_;

    int bIdx = btabStack_.back();
    e.link = btab_[bIdx].last;

    if (adrOverride  $\geq$  0) {
        e.adr = adrOverride;
    } else {
        bool needsOffset = (obj ==
ObjectKind::Variable || obj == ObjectKind::Parameter
|| obj == ObjectKind::VarParam || obj ==
ObjectKind::Field);
        if (needsOffset) {
            e.adr = addrCounter_.back();
            int sz = getTypeSize(type);
            addrCounter_.back() += (sz > 0 ? sz : 1);
        } else {
```

```

        e.adr = 0;
    }
}

int idx = static_cast<int>(tab_.size());
tab_.push_back(e);
btab_[bIdx].last = idx;

return idx;
}

```

Fungsi *declare()* mendaftarkan identifier baru ke scope saat ini. Redclaration dalam scope yang sama terdeteksi melalui *lookupCurrentScope()* dan menyebabkan pengembalian nilai *-1*. Field *link* diisi dengan indeks entry terakhir pada blok aktif sebelum entry baru ini ditambahkan, sehingga membentuk *linked list* terbalik per blok. Field *btab[bIdx].last* kemudian diperbarui ke indeks entry baru agar entry berikutnya dapat melanjutkan *linked list* dengan benar. Offset alamat (*adr*) dihitung secara otomatis berdasarkan ukuran tipe dan diakumulasikan pada *addrCounter_* milik level saat ini.

Fungsi *lookup()* melalui *linked list*

```

int SymbolTable::lookup(const std::string& name)
const {
    for (auto it = btabStack_.rbegin(); it !=
btabsStack_.rend(); ++it) {
        int found = lookupInBlock(name, *it);
        if (found != -1) {
            return found;
        }
    }
    return -1;
}

```

```

int SymbolTable::lookupInBlock(const std::string&
name, int blockIdx) const {
    if (blockIdx < 0 || blockIdx ≥
static_cast<int>(btab_.size())) return -1;
    std::string key = normalize(name);
    int current = btab_[blockIdx].last;
    int guard = 0;
    while (current > 0 && current <
static_cast<int>(tab_.size())) {
        if (normalize(tab_[current].identifier) =
key) {
            return current;
        }
        current = tab_[current].link;
        if (++guard > static_cast<int>(tab_.size()))
{
            break;
        }
    }
    return -1;
}

```

Fungsi *lookup()* mencari identifier dari scope terdalam ke scope global dengan menelusuri *btabStack_* secara terbalik. Untuk setiap blok, *lookupInBlock()* mengikuti *linked list* yang dibentuk oleh field *link*. Pencarian bersifat *case-insensitive* melalui fungsi *normalize()* sesuai konvensi Pascal. Pendekatan ini secara otomatis mengimplementasikan *shadowing*: identifier di scope dalam yang bernama sama dengan scope luar akan ditemukan lebih dahulu karena blok terdalam berada di ujung *btabStack_*.

Fungsi *enterBlock()* dan *leaveBlock()*

```

int SymbolTable::enterBlock() {
    currentLevel_++;

    scopeStart_.push_back(static_cast<int>(tab_.size()));
    addrCounter_.push_back(0);
}

```

```

    BTabEntry b;
    b.last = 0;
    b.lpar = 0;
    b.psize = 0;
    b.vsize = 0;
    int bIdx = static_cast<int>(btab_.size());
    btab_.push_back(b);
    btabStack_.push_back(bIdx);
    return bIdx;
}

void SymbolTable::leaveBlock() {
    if (currentLevel_ ≤ 0) return;
    int bIdx = btabStack_.back();
    if (static_cast<int>(tab_.size()) >
scopeStart_.back()){
        btab_[bIdx].last =
static_cast<int>(tab_.size()) - 1;
    }
    btab_[bIdx].vsize = addrCounter_.back();
    scopeStart_.pop_back();
    addrCounter_.pop_back();
    btabStack_.pop_back();
    currentLevel_--;
}

```

Saat `enterBlock()` dipanggil, entry baru ditambahkan ke `btab`, level leksikal dinaikkan, dan counter alamat baru (dimulai dari 0) di-push ke `addrCounter_` untuk menghitung offset variabel lokal secara independen per blok. Saat `leaveBlock()` dipanggil, total ukuran variabel lokal disimpan ke `btab[bIdx].vsize` sebelum semua stack di-pop. Indeks blok yang aktif dilacak oleh `btabStack_`, bukan oleh `currentLevel_` secara langsung, sehingga indeks `btab` tetap benar meskipun blok untuk prosedur predefined dan record juga ditambahkan ke `btab` di luar siklus `enterBlock/leaveBlock` normal.

2.5. Kelas ASTBuilder

Kelas `ASTBuilder` mengonversi Parse Tree dari Milestone 2 menjadi AST yang lebih singkat. Konversi dilakukan dengan memanggil `build()` yang meneruskan ke fungsi-fungsi `build*()` berdasarkan nama node Parse Tree.

Fungsi `buildVarDeclaration()` untuk handling multi-identifier

```
ASTNode* ASTBuilder::buildVarDeclaration(ParseNode*
node) {
    ASTNode* declarations = new
ASTNode(ASTKind::DeclarationPart);
    for (size_t i = 0; i + 2 < node->children.size();
++i) {
        if (node->children[i]->name !=
"<identifier-list>") continue;

        std::vector<std::string> identifiers =
collectIdentifiers(node->children[i]);
        ASTNode* typeNode =
buildType(node->children[i + 2]);
        for (const std::string& identifier :
identifiers) {
            ASTNode* varDecl = new
ASTNode(ASTKind::VarDecl, identifier);
            varDecl->addChild(clone(typeNode));
            declarations->addChild(varDecl);
        }
        delete typeNode;
    }
    return declarations;
}
```

Fungsi *buildVarDeclaration()* menangani kasus deklarasi beberapa variabel sekaligus dalam satu baris seperti *var a, b, c: integer;*. Fungsi ini mengekstrak seluruh identifier dari *<identifier-list>* menggunakan *collectIdentifiers()*, lalu untuk setiap identifier membuat node *VarDecl* terpisah dengan salinan node tipe (*clone(typeNode)*). Pendekatan ini memastikan setiap variabel memiliki node tipe independen di AST sehingga SemanticAnalyzer dapat mendekorasi masing-masing secara terpisah

Fungsi `buildExpression()`

```

ASTNode* ASTBuilder::buildExpression(ParseNode* node) {
    if (!node) return nullptr;

    if (node->name == "<variable>") return buildVariable(node);
    if (node->name == "<procedure/function-call>") {
        ASTNode* funcCall = buildProcedureCall(node);
        funcCall->kind = ASTKind::FunctionCall;
        return funcCall;
    }
    if (node->name == "<constant>") return buildConstant(node);

    std::string kind = extractTokenKind(node->name);
    if (kind == "IDENT") return new ASTNode(ASTKind::Var,
extractTokenValue(node->name));
    if (kind == "INTCON" || kind == "REALCON" || kind == "STRING" || kind
== "CHARCON") {
        return new ASTNode(ASTKind::Literal,
extractTokenValue(node->name));
    }

    if (!node->children.empty() && (isToken(node->children[0], "PLUS") ||
isToken(node->children[0], "MINUS") || isToken(node->children[0],
"NOTSY")))) {
        ASTNode* unary = new ASTNode(ASTKind::UnaryOp,
operatorText(node->children[0]));
        for (size_t i = 1; i < node->children.size(); ++i) {
            ASTNode* child = buildExpression(node->children[i]);
            if (child && child->kind != ASTKind::Empty) {
                unary->addChild(child);
                return unary;
            }
            delete child;
        }
        return unary;
    }

    ASTNode* current = nullptr;
    for (ParseNode* child : node->children) {
        if (isOperatorWrapper(child)) {
            continue;
        }

        ASTNode* operand = buildExpression(child);
        if (!operand || operand->kind == ASTKind::Empty) {

```

```

        delete operand;
        continue;
    }

    if (!current) {
        current = operand;
        continue;
    }

    auto opIt = std::find_if(node->children.begin(), node->children.end(),
[child](ParseNode* candidate) {
        return candidate == child;
    });
    std::string op = "";
    if (opIt != node->children.begin()) {
        for (auto it = opIt; it != node->children.end(); ++it) {
            if ((*it)->name == "<relational-operator>" ||
                (*it)->name == "<additive-operator>" ||
                (*it)->name == "<multiplicative-operator>") {
                op = operatorText(*it);
                break;
            }
        }
    }

    ASTNode* binOp = new ASTNode(ASTKind::BinOp, op.empty() ? "?" :
op);
    binOp->addChild(current);
    binOp->addChild(operand);
    current = binOp;
}

if (current) return current;
return new ASTNode(ASTKind::Empty);
}

```

Fungsi *buildExpression()* membangun node AST dari node Parse Tree yang merepresentasikan ekspresi. Fungsi ini bersifat rekursif dan menangani berbagai bentuk ekspresi dengan beberapa cabang:

Cabang terminal: Jika node adalah <variable>, <procedure/function-call>, <constant>, IDENT, atau literal (INTCON, REALCON, STRING, CHARCON),

node langsung dikonversi menjadi node AST yang sesuai (Var, FunctionCall, Literal, dst.) tanpa pemrosesan lebih lanjut.

Cabang unary operator: Jika child pertama node adalah PLUS, MINUS, atau NOTSY, dibuat node UnaryOp dengan operator tersebut. Child berikutnya diproses rekursif dan menjadi operand dari node unary tersebut.

Cabang biner (utama): Untuk node ekspresi yang mengandung beberapa operand dan operator, fungsi menelusuri seluruh children secara berurutan. Setiap non-operator-wrapper diproses rekursif menjadi operand. Ketika operand kedua ditemukan, fungsi mencari operator yang mendahuluinya dengan menelusuri balik children hingga menemukan node <relational-operator>, <additive-operator>, atau <multiplicative-operator>. Node BinOp kemudian dibuat dengan operand kiri (current) dan operand kanan yang baru, lalu hasilnya menjadi current baru. Proses ini berulang sehingga menghasilkan pohon biner asosiatif kiri.

2.6. Kelas *SemanticAnalyzer*

Kelas *SemanticAnalyzer* adalah inti analisis semantic Milestone 3. Fungsi *analyze()* menjadi titik masuk yang menginisialisasi simbol predefined lalu memanggil *visit()* pada node root AST.

Fungsi *visit()* sebagai dispatcher utama

```
void SemanticAnalyzer::visit(ASTNode* node) {
    if (!node) {
        return;
    }

    annotate(node);

    switch (node->kind) {
        case ASTKind::Program:
            visitProgram(node);
            break;
        case ASTKind::DeclarationPart:
            visitDeclarationPart(node);
            break;
        case ASTKind::Block:
            visitBlock(node);
            break;
    }
```

```

case ASTKind::ConstDecl:
    visitConstDecl(node);
    break;
case ASTKind::TypeDecl:
    visitTypeDecl(node);
    break;
case ASTKind::VarDecl:
case ASTKind::FieldDecl:
    visitVarDecl(node);
    break;
case ASTKind::ProcDecl:
    visitProcDecl(node);
    break;
case ASTKind::FuncDecl:
    visitFuncDecl(node);
    break;
case ASTKind::Param:
    visitParam(node);
    break;
case ASTKind::Assign:
    visitAssign(node);
    break;
case ASTKind::If:
    visitIf(node);
    break;
case ASTKind::While:
    visitWhile(node);
    break;
case ASTKind::Repeat:
    visitRepeat(node);
    break;
case ASTKind::For:
    visitFor(node);
    break;
case ASTKind::Case:
    visitCase(node);
    break;
case ASTKind::ProcedureCall:

```

```

        visitProcedureCall(node);
        break;
    case ASTKind::FunctionCall:
        visitFunctionCall(node);
        break;
    case ASTKind::BinOp:
        visitBinOp(node);
        break;
    case ASTKind::UnaryOp:
        visitUnaryOp(node);
        break;
    case ASTKind::Literal:
        visitLiteral(node);
        break;
    case ASTKind::Var:
        visitVar(node);
        break;
    case ASTKind::ArrayAccess:
        visitArrayAccess(node);
        break;
    case ASTKind::FieldAccess:
        visitFieldAccess(node);
        break;
    default:
        break;
    }
}

```

Fungsi *visit()* mendispatch setiap node AST ke fungsi *visit*()* yang sesuai. Sebelum dispatch, fungsi *annotate()* dipanggil untuk mengisi *lexicalLevel* dan *blockIndex* pada node dengan nilai saat ini dari SymbolTable.

Fungsi *visitBinOp()* dan *visitAssign()*

```

void SemanticAnalyzer::visitBinOp(ASTNode* node) {
    if (node->children.size() < 2) return;

```

```

        ASTNode* left = node→children[0];
        ASTNode* right = node→children[1];
        visit(left);
        visit(right);

        SemanticType resultType =
inferBinary(node→value, left→inferredType,
right→inferredType);
        if (resultType.kind == TypeKind::Unknown &&
left→inferredType.kind ≠ TypeKind::Unknown &&
right→inferredType.kind ≠ TypeKind::Unknown) {

reportError(SemanticErrors::operatorMismatch(node→va
lue, left→inferredType, right→inferredType));
        }
        node→inferredType = resultType;
    }

void SemanticAnalyzer::visitAssign(ASTNode* node) {
    if (node→children.size() < 2) return;

    ASTNode* target = node→children[0];
    ASTNode* value = node→children[1];

    visit(target);
    visit(value);

    if (target→inferredType.kind ≠
TypeKind::Unknown && value→inferredType.kind ≠
TypeKind::Unknown) {
        if
(!isAssignmentCompatible(target→inferredType,
value→inferredType)) {

reportError(SemanticErrors::typeMismatchAssignment(ta
rget→inferredType, value→inferredType));
        }
    }
}

```

Fungsi *visitBinOp()* terlebih dahulu mengunjungi kedua operand secara rekursif sehingga tipe keduanya sudah diketahui, kemudian memanggil *inferBinary()* untuk menghitung tipe hasil. Error hanya dilaporkan jika hasil *Unknown* tetapi kedua operand *bukan Unknown*, sehingga error pada sub-ekspresi sebelumnya tidak menghasilkan error ganda. Fungsi *visitAssign()* menggunakan pola yang sama: kunjungi sisi kiri dan kanan, lalu periksa kompatibilitas assignment.

Fungsi *visitProcDecl()* dan *visitFuncDecl()*

```
void SemanticAnalyzer::visitProcDecl(ASTNode* node) {
    annotate(node);
    SemanticType type =
makeType(TypeKind::Procedure);
    node→inferredType = type;
    node→tabIndex = symbols→declare(node→value,
ObjectKind::Procedure, type, 1, 0);

    if (node→tabIndex == -1) {
        reportError("Redeclaration of identifier '" +
node→value + "'");
        return;
    }

    int blockIndex = symbols→enterBlock();
    symbols→tabAt(node→tabIndex).ref = blockIndex;
    node→blockIndex = blockIndex;
    node→lexicalLevel = symbols→currentLevel();

    for (ASTNode* child : node→children) {
        if (!child) {
            continue;
        }

        if (child→kind == ASTKind::DeclarationPart
&& child→value == "parameters") {
            visitDeclarationPart(child);
        } else if (child→kind == ASTKind::Block) {
```

```

        visitBlock(child);
    }
}

symbols→leaveBlock();
}

void SemanticAnalyzer::visitFuncDecl(ASTNode* node) {
    annotate(node);
    ASTNode* returnTypeNode = nullptr;
    ASTNode* parameterPart = nullptr;
    ASTNode* body = nullptr;

    for (ASTNode* child : node→children) {
        if (!child) {
            continue;
        }

        if (child→kind == ASTKind::Type) {
            returnTypeNode = child;
        } else if (child→kind ==
ASTKind::DeclarationPart && child→value ==
"parameters") {
            parameterPart = child;
        } else if (child→kind == ASTKind::Block) {
            body = child;
        }
    }

    TypeInfo returnType = returnTypeNode ?
resolveType(returnTypeNode) : TypeInfo{};
    if (!returnTypeNode || returnType.type.kind ==
TypeKind::Unknown) {
        reportError("Invalid function return type for
'" + node→value + "'");
    }

    node→inferredType = returnType.type;
    node→tabIndex = symbols→declare(node→value,

```

```

ObjectKind::Function, returnType.type, 1, 0);

    if (node→tabIndex == -1) {
        reportError("Redeclaration of identifier '" +
node→value + "'");
        return;
    }

    int blockIndex = symbols→enterBlock();
    symbols→tabAt(node→tabIndex).ref = blockIndex;
    node→blockIndex = blockIndex;
    node→lexicalLevel = symbols→currentLevel();

    if (parameterPart) {
        visitDeclarationPart(parameterPart);
    }

    if (body) {
        visitBlock(body);
    }

    if (!body || !alwaysAssignsFunctionResult(body,
node→value)) {
        reportError("Function '" + node→value + "'
does not assign a return value on all control-flow
paths");
    }

    symbols→leaveBlock();
}

```

Saat sebuah prosedur atau fungsi dideklarasikan, *visitProcDecl()* memanggil *symbols→enterBlock()* untuk membuat scope baru dan mendapatkan indeks blok baru pada *btabs*. Indeks ini disimpan ke field *ref* pada entry prosedur di *tab*, sehingga SemanticAnalyzer dapat menemukan blok prosedur saat memeriksa pemanggilan. Setelah seluruh isi prosedur dianalisis, *leaveBlock()* dipanggil untuk menutup scope dan menyimpan ukuran variabel lokal ke *btabs[blockIndex].vsz*. Fungsi *visitFuncDecl()* bekerja dengan cara yang sama, ditambah pemeriksaan

apakah fungsi selalu mengisikan nilai kembalian menggunakan *alwaysAssignsFunctionResult()*.

Fungsi visitArrayAccess()

```
void SemanticAnalyzer::visitArrayAccess(ASTNode*
node) {
    if (node→children.empty()) return;
    ASTNode* arrayNode = node→children[0];
    visit(arrayNode);

    SemanticType arrayType = arrayNode→inferredType;
    if (arrayType.kind ≠ TypeKind::Unknown &&
arrayType.kind ≠ TypeKind::Array) {

reportError(SemanticErrors::notAnArray(arrayNode→val
ue));
        node→inferredType = makeUnknownType();
        return;
    }

    if (arrayType.kind = TypeKind::Array) {
        const ATabEntry& arrayEntry =
symbols→atabAt(arrayType.ref);
        for (size_t i = 1; i <
node→children.size(); i++) {
            ASTNode* indexExpr = node→children[i];
            visit(indexExpr);
            if (indexExpr→inferredType.kind ≠
TypeKind::Unknown && !isCompatible(arrayEntry.xtyp,
indexExpr→inferredType)) {

reportError(SemanticErrors::invalidArrayIndex(arrayEn
try.xtyp, indexExpr→inferredType));
            }
        }
        node→inferredType = arrayEntry.etyt;
    } else {
```

```

        node→inferredType = makeUnknownType();
    }
}

```

Fungsi *visitArrayAccess()* memvalidasi bahwa variabel yang diakses dengan [...] memang bertipe *Array*. Jika ya, informasi tipe indeks (*xtyp*) dan tipe elemen (*etyp*) diambil dari *atab* menggunakan *arrayType.ref* sebagai indeks. Tipe setiap ekspresi indeks kemudian diperiksa kompatibilitasnya terhadap *xtyp*. Tipe hasil akses array diisi dengan *etyp*, yaitu tipe elemen array tersebut.

Fungsi *visitFor()*

```

void SemanticAnalyzer::visitFor(ASTNode* node) {
    // Implementasi untuk ngecek start loop, iterasi
    stop loop (Assignment Compatible dengan start),
    // memastikan variabel index itu ordinal &
    declared var.
    int index = symbols→lookup(node→value);
    SemanticType controlType = makeUnknownType();
    if (index == -1) {

reportError(SemanticErrors::undeclaredIdentifier(node
→value));
    } else {
        node→tabIndex = index;
        controlType = symbols→tabAt(index).type;
        if (!isOrdinal(controlType)) {

reportError(SemanticErrors::invalidForControlVar(node
→value));
        }
    }

    for (size_t i = 0; i < node→children.size();
++i) {
        ASTNode* child = node→children[i];
        visit(child);
    }
}

```

```

        if (i < 2 && controlType.kind ≠
TypeKind::Unknown && child→inferredType.kind ≠
TypeKind::Unknown &&
            !isAssignmentCompatible(controlType,
child→inferredType)) {

reportError(SemanticErrors::typeMismatchAssignment(co
ntrolType, child→inferredType));
        }
    }
}

```

Fungsi *visitFor()* memeriksa tiga hal: variabel kontrol harus sudah dideklarasikan, tipenya harus ordinal (*Integer*, *Char*, *Boolean*, atau *Subrange*), dan kedua ekspresi batas (*start* dan *stop*) harus kompatibel secara assignment dengan tipe variabel kontrol. Pemeriksaan batas hanya berlaku untuk dua child pertama (indeks 0 dan 1), sedangkan child berikutnya adalah body loop yang cukup dikunjungi tanpa pemeriksaan tipe tambahan.

2.7. Namespace *SemanticErrors*

Namespace *SemanticErrors* pada *SemanticErrors.hpp/cpp* menyediakan fungsi-fungsi pembuat pesan kesalahan semantis yang terstandarisasi. Setiap fungsi menerima informasi konteks dan mengembalikan *std::string* dengan pesan yang deskriptif:

```

namespace SemanticErrors {
    std::string undeclaredIdentifier(const
std::string& name);
    std::string typeMismatchAssignment(const
SemanticType& target, const SemanticType& value);
    std::string operatorMismatch(const std::string&
op, const SemanticType& left, const SemanticType&
right);
    std::string operatorMismatchUnary(const
std::string& op, const SemanticType& operand);
    std::string conditionNotBoolean(const
std::string& context);
}

```

```

        std::string notAnArray(const std::string& name);
        std::string invalidArrayIndex(const SemanticType&
expected, const SemanticType& actual);
        std::string notARecord(const std::string& name);
        std::string invalidRecordField(const std::string&
recordName, const std::string& fieldName);
        std::string notAProcedure(const std::string&
name);
        std::string notAFunction(const std::string&
name);
        std::string invalidArgumentCount(const
std::string& name, int expected, int actual);
        std::string invalidArgumentType(const
std::string& name, int paramIndex, const
SemanticType& expected, const SemanticType& actual);
        std::string invalidForControlVar(const
std::string& name);
    }

```

Pemisahan pesan error ke namespace tersendiri memudahkan konsistensi format pesan di seluruh codebase. Setiap pemanggilan fungsi error di *SemanticAnalyzer* cukup memanggil fungsi yang sesuai dari namespace ini, sehingga perubahan format pesan hanya perlu dilakukan di satu tempat.

2.8. Kelas ASTPrinter

Kelas *ASTPrinter* mencetak DAST ke output stream dengan format visual hierarkis. Setiap node dicetak dengan label *kindToString(kind)*, nilai identifier (jika ada), dan atribut dekorasi *type*, *tab*, *block*, serta *level*:

```

void ASTPrinter::print(ASTNode* root, std::ostream&
out, const std::string& prefix, bool isLast, bool
isRoot) {
    if (!root) return;

    if (!isRoot) {
        out << prefix << (isLast ? "└─ " : "├─ ");
    } else {

```

```

        out << prefix;
    }

    out << kindToString(root→kind);
    if (!root→value.empty()) out << "(" <<
root→value << ")";

    out << " [type=" <<
typeKindToString(root→inferredType.kind)
        << ", tab=" << root→tabIndex
        << ", block=" << root→blockIndex
        << ", level=" << root→lexicalLevel << "]\n";

    std::string childPrefix = prefix;
    if (!isRoot) {
        childPrefix += (isLast ? "    " : "|    ");
    }

    for (size_t i = 0; i < root→children.size();
++i) {
        print(root→children[i], out, childPrefix, (i
= root→children.size() - 1), false);
    }
}

```

Format output menggambarkan informasi dekorasi lengkap per node. Contohnya, node *BinOp(+)* akan dicetak sebagai *BinOp(+)* [type=Integer, tab=-1, block=0, level=0], menunjukkan bahwa tipe hasil inferensi adalah *Integer*, node ini tidak terdaftar langsung di *tab* (karena bukan identifier), berada di block 0, dan level leksikal 0.

2.9. Main.cpp

Entry point program mengintegrasikan seluruh komponen pipeline. File input dan output dibaca dari argumen command-line, dengan folder test yaitu *test/milestone-3/*:

```

Lexer lexer(inputFile);

```

```

Parser parser(lexer);
ParseNode* parseRoot = parser.parse();

if (parser.isSuccess()) {
    ASTBuilder builder;
    ASTNode* astRoot = builder.build(parseRoot);

    SemanticAnalyzer analyzer;
    bool semanticSuccess = analyzer.analyze(astRoot);

    out << "Semantic Analysis: " << (semanticSuccess
? "SUCCESS" : "FAILED") << "\n\n";
    if (!semanticSuccess) {
        out << "Semantic Errors:\n";
        for (const SemanticError& e :
analyzer.errors())
            out << "- " << e.message << "\n";
    }
    out << "Decorated AST:\n";
    ASTPrinter printer;
    printer.print(astRoot, out);
    out << "\n";
    analyzer.symbolTable().printTables(out);
    delete astRoot;
}
delete parseRoot;

```

Jika analisis sintaks berhasil, ASTBuilder mengkonversi Parse Tree menjadi AST, kemudian SemanticAnalyzer menganalisis AST tersebut. Output file berisi status analisis semantic, daftar error semantic jika ada, DAST yang sudah didekorasi, serta isi ketiga tabel simbol (tab, btab, atab). Memori AST dan Parse Tree dibebaskan secara eksplisit setelah seluruh output ditulis.

2.10. Semantic Action pada Syntax-Directed Translation

Konversi *Parse Tree* menjadi AST dilakukan menggunakan *Syntax-Directed Translation Scheme* (SDTS). Setiap aturan produksi grammar dilengkapi dengan *semantic action*, yaitu tindakan yang dieksekusi saat node *Parse Tree* yang bersangkutan diproses oleh *ASTBuilder*. Semantic action bertanggung jawab

untuk membentuk node AST yang baru, menghilangkan informasi sintaksis yang tidak diperlukan seperti *BEGIN*, *END*, tanda kurung, dan *SEMICOLON*, serta menyerahkan node dari bawah ke atas (*bottom-up*) agar node induk dapat merakitnya menjadi satu kesatuan AST yang utuh.

Tabel berikut menuliskan semantic action yang digunakan pada setiap aturan produksi utama:

Tabel 6 Semantic Action per Aturan Produksi

Aturan Produksi	Semantic Action	Node AST yang Dihasilkan
<program>	Ekstrak nama program dari token IDENT pada <program-header>. Kunjungi <declaration-part> dan <compound-statement> sebagai anak.	Program(nama)
<var-declaration>	Untuk setiap <identifier-list>, kumpulkan semua IDENT via collectIdentifiers(). Buat satu VarDecl per identifier dengan salinan node tipe yang sama (clone).	VarDecl(nama) [satu per identifier]
<const-declaration>	Untuk setiap pasangan IDENT = <constant>, buat node ConstDecl dengan nama identifier dan node Literal sebagai anak.	ConstDecl(nama)
<type-declaration>	Untuk setiap pasangan IDENT = <type>, buat node TypeDecl dengan nama identifier dan node tipe sebagai anak.	TypeDecl(nama)
<array-type>	Buat node ArrayType dengan dua anak: node RangeType (dari <range>) dan node tipe elemen (dari <type> setelah OF).	ArrayType
<record-type>	Untuk setiap <field-part>, buat node FieldDecl per identifier field. Seluruh FieldDecl menjadi anak node RecordType.	RecordType
<procedure-declaration>	Buat node ProcDecl dengan nama prosedur. Kunjungi <formal-parameter-list> (jika ada)	ProcDecl(nama)

	dan <block> sebagai anak.	
<function-declaration>	Buat node FuncDecl dengan nama fungsi. Kunjungi parameter, tipe kembalian (IDENT setelah COLON dibungkus node Type), dan <block>.	FuncDecl(nama)
<assignment-statement>	Buat node Assign. Anak pertama: buildVariable(<variable>). Anak kedua: buildExpression(<expression>). Token BECOMES dibuang.	Assign
<if-statement>	Buat node If. Anak: buildExpression(<expression>), buildStatement(<statement> then), buildStatement(<statement> else, jika ada).	If
<while-statement>	Buat node While. Anak: buildExpression(<expression>), buildBlock(<compound-statement>). Token WHILE dan DO dibuang.	While
<repeat-statement>	Buat node Repeat. Anak pertama: Block berisi seluruh statement. Anak terakhir: buildExpression(<expression> setelah UNTIL).	Repeat
<for-statement>	Buat node For dengan value = nama variabel kontrol (IDENT). Anak: ekspresi batas bawah, ekspresi batas atas, body. Token FOR, BECOMES, TO/DOWNT, DO dibuang.	For(var)
<case-statement>	Buat node Case. Anak pertama: ekspresi selektor. Setiap <case-block> dikonversi menjadi node Case dengan label konstanta dan statement sebagai anak.	Case
<expression> / <simple-expression> / <term>	Ratakan hierarki ekspresi menjadi pohon biner. Untuk unary (PLUS/MINUS/NOTSY di child pertama): buat UnaryOp. Untuk biner: iterasi children, bentuk	BinOp(op) / UnaryOp(op) / Literal / Var

	BinOp(op, left, right) secara asosiatif kiri dengan menelusuri balik children untuk menemukan operator.	
<variable>	Buat node Var dari IDENT pertama. Untuk setiap <component-variable> berikutnya: jika LBRACK → buat ArrayAccess; jika PERIOD → buat FieldAccess(field). Susun secara berantai.	Var / ArrayAccess / FieldAccess
<procedure/function-call>	Buat node ProcedureCall (atau FunctionCall jika dipanggil dari ekspresi) dengan nama prosedur/fungsi. Setiap argumen dari <parameter-list> dikonversi via buildExpression() menjadi anak.	ProcedureCall(nama) / FunctionCall(nama)
<constant>	Buat node Literal dengan nilai token (INTCON, REALCON, CHARCON, STRING). Tanda kutip CHARCON/STRING dibuang.	Literal(nilai)

2.11. Proses Dekorasi AST (Type & Scope Checking)

Setelah AST selesai dibangun, *SemanticAnalyzer* menelusurinya secara *Depth-First Search* (DFS) menggunakan fungsi *visit()*. Sebelum penelusuran dimulai, *SemanticAnalyzer* memanggil *ensurePredefinedSymbols()* yang memanggil *symbols->initPredefined()* untuk mengisi *SymbolTable* dengan identifier bawaan: tipe dasar (*Integer*, *Real*, *Char*, *Boolean*, *String*), konstanta boolean (*True* = 1, *False* = 0), serta prosedur I/O bawaan (*writeln*, *readln*, *write*, *read*). Seluruh entry ini didaftarkan di level leksikal 0 sehingga dapat ditemukan dari scope manapun.

Tabel 7 Daftar Predefined Identifier

Identifier	ObjectKind	Type (kind)	adr	Keterangan
Integer	Type	Integer	0	Tipe dasar bilangan bulat
Real	Type	Real	0	Tipe dasar bilangan real

Char	Type	Char	0	Tipe dasar karakter
Boolean	Type	Boolean	0	Tipe dasar boolean
String	Type	String	0	Tipe dasar string
True	Constant	Boolean	1	Nilai ordinal 1; initialized = true
False	Constant	Boolean	0	Nilai ordinal 0; initialized = true
writeln	Procedure	Void	0	Prosedur output dengan newline; ref ke btab
readln	Procedure	Void	0	Prosedur input dengan newline; ref ke btab
write	Procedure	Void	0	Prosedur output tanpa newline; ref ke btab
read	Procedure	Void	0	Prosedur input tanpa newline; ref ke btab

Prosedur bawaan (*writeln*, *readln*, *write*, *read*) masing-masing memiliki entry blok tersendiri di *btab* yang dibuat saat *initPredefined()* dipanggil. Field *ref* pada entry *tab* prosedur tersebut menunjuk ke indeks blok yang bersangkutan di *btab*. Karena keempat prosedur ini bersifat *variadic* (jumlah argumen tidak tetap), tidak ada entry parameter yang didaftarkan ke dalam bloknya.

a. Manajemen Scope (Push dan Pop)

Setiap kali SemanticAnalyzer memasuki blok baru (prosedur, fungsi, atau tipe record), fungsi *symbols->enterBlock()* dipanggil untuk melakukan *push* scope baru. Fungsi ini membuat entry baru di *btab*, menaikkan *currentLevel_*, dan memulai counter alamat baru dari 0 untuk blok tersebut. Saat keluar dari blok, *symbols->leaveBlock()* melakukan *pop* scope, menyimpan total ukuran variabel lokal ke *btab[bldx].vsze*, lalu menurunkan *currentLevel_*. Pola ini diimplementasikan di *visitProcDecl()*, *visitFuncDecl()*, dan *resolveRecordType()*:

```
int blockIndex = symbols->enterBlock();
symbols->tabAt(node->tabIndex).ref = blockIndex;
visitDeclarationPart(parameterPart);
visitBlock(body);
symbols->leaveBlock();
```

b. Registrasi Simbol

Setiap kali ditemukan deklarasi identifiier, SemanticAnalyzer memanggil *symbols->declare()* untuk mendaftarkan identifiier tersebut ke *tab* beserta seluruh atributnya: nama, kelas objek (*ObjectKind*), tipe data, level leksikal, dan offset alamat memori. Jika *declare()* mengembalikan *-1*, berarti terjadi redeclaration dalam scope yang sama dan error dilaporkan. Berikut contoh registrasi untuk masing-masing jenis deklarasi:

Tabel 8 Contoh Registrasi

Jenis Deklarasi	Pemanggilan declare()
<i>visitVarDecl()</i>	<i>symbols->declare(name, Variable, type)</i>
<i>visitConstDecl()</i>	<i>symbols->declare(name, Constant, type, 1, ordinal); tabAt(idk).initialized = true</i>
<i>visitTypeDecl()</i>	<i>symbols->declare(name, Type, type, 1, 0)</i>
<i>visitProcDecl()</i>	<i>symbols->declare(name, Procedure, makeType(Procedure), 1, 0)</i>
<i>visitFuncDecl()</i>	<i>symbols->declare(name, Function, returnType, 1, 0)</i>
<i>visitParam()</i>	<i>symbols->declare(name, Parameter, type)</i>

c. Lookup Identifier

Setiap kali ditemukan penggunaan identifier dalam program, SemanticAnalyzer memanggil *symbols->lookup()* untuk mencarinya dari scope terdalam ke scope global. Lookup dilakukan melalui *linked list* per blok yang dibentuk oleh field *link* pada *tab*. Jika identifier tidak ditemukan, error *undeclaredIdentifier* dilaporkan dan tipe node diset *Unknown* agar pemeriksaan berikutnya tidak menghasilkan error ganda. Jika ditemukan, indeks entri disimpan ke *node->tabIndex* dan tipe identifier diambil dari *symbols->tabAt(index).type* untuk mengisi *node->inferredType*:

```
void SemanticAnalyzer::visitVar(ASTNode* node) {
    int index = symbols->lookup(node->value);
    if (index == -1) {

reportError(SemanticErrors::undeclaredIdentifier
(node->value));
        node->inferredType = makeUnknownType();
        return;
    }
    node->tabIndex = index;
    node->inferredType =
symbols->tabAt(index).type;
}
```

d. Inferensi dan Validasi Tipe Data

Penelusuran dilakukan secara top-down menggunakan fungsi *visit()* yang dipanggil rekursif mulai dari node root. Namun pada setiap node ekspresi, *visit()* terlebih dahulu dipanggil pada node anak sebelum tipe node induk dihitung, sehingga informasi tipe mengalir dari bawah ke atas (*synthesized attributes*) Terdapat dua jenis pemeriksaan:

- **Inferensi tipe operasi:** fungsi *inferBinary()* dan *inferUnary()* dari modul *TypeSystem* menghitung tipe hasil berdasarkan operator dan tipe operand. Contohnya $Integer + Integer \rightarrow Integer$, $Integer /$

Integer $\rightarrow$ *Real*, dan operator relasional selalu menghasilkan *Boolean*.

- **Validasi kompatibilitas tipe:** fungsi *isAssignmentCompatible()* digunakan pada *visitAssign()* untuk memastikan tipe nilai yang diberikan kompatibel dengan tipe variabel tujuan. Aturan yang berlaku antara lain *Real* := *Integer* valid (widening), namun *Integer* := *Real* tidak valid.

Jika hasil inferensi adalah *Unknown* tetapi kedua operand bukan *Unknown*, error dilaporkan. Mekanisme ini mencegah pelaporan error ganda akibat error pada sub-ekspresi yang sudah dilaporkan sebelumnya.

e. Anotasi Node AST

Setelah seluruh pemeriksaan dilakukan, setiap node AST didekorasi dengan tiga informasi tambahan menggunakan fungsi *annotate()*:

- **inferredType:** tipe data akhir dari ekspresi atau identifier pada node tersebut, dihasilkan dari inferensi tipe atau lookup ke *tab*.
- **tabIndex:** indeks entri pada *tab* yang merujuk ke deklarasi identifier terkait. Bernilai -1 untuk node yang tidak merujuk identifier (seperti *BinOp* atau *Literal*).
- **blockIndex** dan **lexicalLevel:** indeks blok pada *btabs* dan level leksikal saat node tersebut berada, diisi dari nilai *symbols->currentBlock()* dan *symbols->currentLevel()* pada saat node dikunjungi.

```
void SemanticAnalyzer::annotate(ASTNode* node) {
    if (!node) return;
    node->lexicalLevel =
symbols->currentLevel();
    node->blockIndex =
symbols->currentBlock();
}
```

Hasil akhir dari seluruh proses ini adalah *Decorated AST* (DAST) di mana setiap node sudah memiliki informasi tipe, referensi ke tabel simbol, dan

level leksikal yang lengkap. DAST ini kemudian dicetak oleh *ASTPrinter* bersama isi ketiga tabel simbol (*tab*, *btab*, *atab*) ke dalam file output.

BAB 3 PENGUJIAN

1. Test Case 1

input1.txt
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>

output1.txt

Semantic Analysis: SUCCESS

Decorated AST:

Program(Hello) [type=Void, tab=44, block=0, level=0]

├─ Declarations [type=Unknown, tab=-1, block=0, level=0]

│ ├── VarDecl(a) [type=Integer, tab=45, block=0, level=0]

│ │ └─ Type(integer) [type=Integer, tab=33, block=0, level=0]

│ ├── VarDecl(b) [type=Integer, tab=46, block=0, level=0]

│ │ └─ Type(integer) [type=Integer, tab=33, block=0, level=0]

├─ Block [type=Unknown, tab=-1, block=0, level=0]

│ ├── Assign [type=Unknown, tab=-1, block=0, level=0]

│ │ ├── Var(a) [type=Integer, tab=45, block=0, level=0]

│ │ └─ Literal(5) [type=Integer, tab=-1, block=0, level=0]

│ ├── Assign [type=Unknown, tab=-1, block=0, level=0]

│ │ ├── Var(b) [type=Integer, tab=46, block=0, level=0]

│ │ ├── BinOp(+) [type=Integer, tab=-1, block=0, level=0]

│ │ │ ├── Var(a) [type=Integer, tab=45, block=0, level=0]

│ │ │ └─ Literal(10) [type=Integer, tab=-1, block=0, level=0]

│ ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]

│ │ ├── Literal(Result =) [type=String, tab=-1, block=0, level=0]

│ │ └─ Var(b) [type=Integer, tab=46, block=0, level=0]

│ └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

Symbol Table - tab:

idx	identifir	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no

38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	Hello	Program	Void	0	1	0	0	43	no
45	a	Variable	Integer	0	1	0	0	44	no
46	b	Variable	Integer	0	1	0	1	45	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	46	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

2. Test Case 2

input2.txt
<pre> program ValidControlFlow; var i, best, total: integer; flag: boolean; scores: array [1..5] of integer; begin i := 1; best := 0; total := 0; flag := true; while i ≤ 5 do begin scores[i] := i * 10; total := total + scores[i]; if scores[i] > best then best := scores[i] </pre>

```

    else
        best := best;

    i := i + 1;
end;

for i := 1 to 5 do
begin
    total := total + i;
end;

repeat
    total := total - 1;
until total ≤ 100;

if flag and (best > 0) then
    writeln('Best score = ', best)
else
    writeln('No score');
end.

```

output2.txt

Semantic Analysis: SUCCESS

Decorated AST:

```

Program(ValidControlFlow) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── VarDecl(i) [type=Integer, tab=45, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── VarDecl(best) [type=Integer, tab=46, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── VarDecl(total) [type=Integer, tab=47, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── VarDecl(flag) [type=Boolean, tab=48, block=0, level=0]
│   │   └── Type(boolean) [type=Boolean, tab=36, block=0, level=0]
│   └── VarDecl(scores) [type=Array, tab=49, block=0, level=0]
│       ├── ArrayType [type=Array, tab=-1, block=0, level=0]
│       │   ├── RangeType [type=Subrange, tab=-1, block=0, level=0]
│       │   │   ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│       │   │   └── Literal(5) [type=Integer, tab=-1, block=0, level=0]
│       │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
└── Block [type=Unknown, tab=-1, block=0, level=0]
    ├── Assign [type=Unknown, tab=-1, block=0, level=0]
    │   ├── Var(i) [type=Integer, tab=45, block=0, level=0]
    │   └── Literal(1) [type=Integer, tab=-1, block=0, level=0]

```

```

├─ Assign [type=Unknown, tab=-1, block=0, level=0]
│   └─ Var(best) [type=Integer, tab=46, block=0, level=0]
│       └─ Literal(0) [type=Integer, tab=-1, block=0, level=0]
├─ Assign [type=Unknown, tab=-1, block=0, level=0]
│   └─ Var(total) [type=Integer, tab=47, block=0, level=0]
│       └─ Literal(0) [type=Integer, tab=-1, block=0, level=0]
├─ Assign [type=Unknown, tab=-1, block=0, level=0]
│   └─ Var(flag) [type=Boolean, tab=48, block=0, level=0]
│       └─ Var(true) [type=Boolean, tab=38, block=0, level=0]
├─ While [type=Unknown, tab=-1, block=0, level=0]
│   └─ BinOp(≤) [type=Boolean, tab=-1, block=0, level=0]
│       └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│           └─ Literal(5) [type=Integer, tab=-1, block=0, level=0]
│   └─ Block [type=Unknown, tab=-1, block=0, level=0]
│       └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│           └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│               └─ Var(scores) [type=Array, tab=49, block=0, level=0]
│           └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│           └─ BinOp(*) [type=Integer, tab=-1, block=0, level=0]
│               └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│                   └─ Literal(10) [type=Integer, tab=-1, block=0, level=0]
│       └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│           └─ Var(total) [type=Integer, tab=47, block=0, level=0]
│           └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│               └─ Var(total) [type=Integer, tab=47, block=0, level=0]
│                   └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│                       └─ Var(scores) [type=Array, tab=49, block=0, level=0]
│                           └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│       └─ If [type=Unknown, tab=-1, block=0, level=0]
│           └─ BinOp(>) [type=Boolean, tab=-1, block=0, level=0]
│               └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│                   └─ Var(scores) [type=Array, tab=49, block=0, level=0]
│                       └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│               └─ Var(best) [type=Integer, tab=46, block=0, level=0]
│           └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│               └─ Var(best) [type=Integer, tab=46, block=0, level=0]
│                   └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│                       └─ Var(scores) [type=Array, tab=49, block=0, level=0]
│                           └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│           └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│               └─ Var(best) [type=Integer, tab=46, block=0, level=0]
│                   └─ Var(best) [type=Integer, tab=46, block=0, level=0]
│       └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│           └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│           └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│               └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│                   └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
│       └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
├─ For(i) [type=Unknown, tab=45, block=0, level=0]
│   └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
│   └─ Literal(5) [type=Integer, tab=-1, block=0, level=0]
│   └─ Block [type=Unknown, tab=-1, block=0, level=0]
│       └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│           └─ Var(total) [type=Integer, tab=47, block=0, level=0]
│           └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│               └─ Var(total) [type=Integer, tab=47, block=0, level=0]
│                   └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│       └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

```

Repeat [type=Unknown, tab=-1, block=0, level=0]
├── Block [type=Unknown, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(total) [type=Integer, tab=47, block=0, level=0]
│   │   └── BinOp(-) [type=Integer, tab=-1, block=0, level=0]
│   │       ├── Var(total) [type=Integer, tab=47, block=0, level=0]
│   │       └── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│   └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
├── BinOp(≤) [type=Boolean, tab=-1, block=0, level=0]
│   ├── Var(total) [type=Integer, tab=47, block=0, level=0]
│   └── Literal(100) [type=Integer, tab=-1, block=0, level=0]
├── If [type=Unknown, tab=-1, block=0, level=0]
│   ├── BinOp(and) [type=Boolean, tab=-1, block=0, level=0]
│   │   ├── Var(flag) [type=Boolean, tab=48, block=0, level=0]
│   │   └── BinOp(>) [type=Boolean, tab=-1, block=0, level=0]
│   │       ├── Var(best) [type=Integer, tab=46, block=0, level=0]
│   │       └── Literal(0) [type=Integer, tab=-1, block=0, level=0]
│   ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
│   │   ├── Literal(Best score = ) [type=String, tab=-1, block=0, level=0]
│   │   └── Var(best) [type=Integer, tab=46, block=0, level=0]
│   ├── ProcedureCall(writeln) [type=Unknown, tab=40, block=0, level=0]
│   │   └── Literal(No score) [type=String, tab=-1, block=0, level=0]
└── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifier	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	ValidControlFlowProgram		Void	0	1	0	0	43	no
45	i	Variable	Integer	0	1	0	0	44	no
46	best	Variable	Integer	0	1	0	1	45	no
47	total	Variable	Integer	0	1	0	2	46	no
48	flag	Variable	Boolean	0	1	0	3	47	no
49	scores	Variable	Array[ref=1]0	1	0	0	4	48	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	49	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size
1	Subrange	Integer	0	1	5	1	5

3. Test Case 3

input3.txt

```
program DeclarationDemo;
const
  limit = 10;
type
  Index = 1..10;
  IntArray = array [Index] of integer;
  Point = record
    x, y: integer;
  end;
var
  arr: IntArray;
  p: Point;
  value: integer;
function AddOne(x: integer): integer;
begin
  AddOne := x + 1;
end;
begin
  value := limit;
end.
```

output3.txt

Semantic Analysis: SUCCESS

Decorated AST:

```
Program(DeclarationDemo) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── ConstDecl(limit) [type=Integer, tab=45, block=0, level=0]
│   │   └── Literal(10) [type=Integer, tab=-1, block=0, level=0]
│   ├── TypeDecl(Index) [type=Subrange, tab=46, block=0, level=0]
│   │   └── RangeType [type=Subrange, tab=-1, block=0, level=0]
│   │       ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│   │       └── Literal(10) [type=Integer, tab=-1, block=0, level=0]
│   ├── TypeDecl(IntArray) [type=Array, tab=47, block=0, level=0]
│   │   ├── ArrayType [type=Array, tab=-1, block=0, level=0]
│   │   │   ├── Type(Index) [type=Subrange, tab=46, block=0, level=0]
│   │   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   └── TypeDecl(Point) [type=Record, tab=50, block=0, level=0]
```

```

└─ RecordType [type=Record, tab=-1, block=5, level=1]
    └─ FieldDecl(x) [type=Integer, tab=48, block=5, level=1]
        └─ Type(integer) [type=Integer, tab=33, block=5, level=1]
    └─ FieldDecl(y) [type=Integer, tab=49, block=5, level=1]
        └─ Type(integer) [type=Integer, tab=33, block=5, level=1]
└─ VarDecl(arr) [type=Array, tab=51, block=0, level=0]
    └─ Type(IntArray) [type=Array, tab=47, block=0, level=0]
└─ VarDecl(p) [type=Record, tab=52, block=0, level=0]
    └─ Type(Point) [type=Record, tab=50, block=0, level=0]
└─ VarDecl(value) [type=Integer, tab=53, block=0, level=0]
    └─ Type(integer) [type=Integer, tab=33, block=0, level=0]
└─ FunctionDecl(AddOne) [type=Integer, tab=54, block=6, level=1]
    └─ Declarations(parameters) [type=Unknown, tab=-1, block=6, level=1]
        └─ Param(x) [type=Integer, tab=55, block=6, level=1]
            └─ Type(integer) [type=Integer, tab=33, block=6, level=1]
    └─ Block [type=Unknown, tab=-1, block=6, level=1]
        └─ Assign [type=Unknown, tab=-1, block=6, level=1]
            └─ Var(AddOne) [type=Integer, tab=54, block=6, level=1]
                └─ BinOp(+) [type=Integer, tab=-1, block=6, level=1]
                    └─ Var(x) [type=Integer, tab=55, block=6, level=1]
                        └─ Literal(1) [type=Integer, tab=-1, block=6, level=1]
            └─ EmptyStatement [type=Unknown, tab=-1, block=6, level=1]
        └─ Type(integer) [type=Integer, tab=33, block=0, level=0]
└─ Block [type=Unknown, tab=-1, block=0, level=0]
    └─ Assign [type=Unknown, tab=-1, block=0, level=0]
        └─ Var(value) [type=Integer, tab=53, block=0, level=0]
        └─ Var(limit) [type=Integer, tab=45, block=0, level=0]
    └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifir	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	DeclarationDemo	Program	Void	0	1	0	0	43	no
45	limit	Constant	Integer	0	1	0	10	44	yes
46	Index	Type	Subrange	0	1	0	0	45	no
47	IntArray	Type	Array[ref=1]	0	1	0	0	46	no
48	x	Field	Integer	0	1	1	0	0	no
49	y	Field	Integer	0	1	1	1	48	no
50	Point	Type	Record[ref=5]	0	1	0	0	47	no
51	arr	Variable	Array[ref=1]	0	1	0	0	50	no
52	p	Variable	Record[ref=5]	0	1	0	1	51	no
53	value	Variable	Integer	0	1	0	2	52	no
54	AddOne	Function	Integer	6	1	0	0	53	no
55	x	Parameter	Integer	0	1	1	0	0	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	54	0	0	0			
1	40	0	0	0			
2	41	0	0	0			
3	42	0	0	0			
4	43	0	0	0			
5	49	0	0	2			
6	55	0	0	1			
Symbol Table - atab:							
idx	xtyp	etyp	eref	low	high	elsz	size

1	Subrange	Integer	0	1	10	1	10

4. Test Case 4

input4.txt
<pre> program InvalidLookupAndAccess; type Point = record x: integer; end; var a: integer; a: real; b: boolean; p: Point; begin missing := 1; a[1] := 3; p.z := 4; if a then b := true; end. </pre>

output4.txt
<pre> Semantic Analysis: FAILED Semantic Errors: - Redeclaration of identifier 'a' - Undeclared identifier 'missing' </pre>

- Identifier 'a' is not an array
- Record 'p' has no field 'z'
- Condition in if statement must be Boolean

Decorated AST:

```

Program(InvalidLookupAndAccess) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── TypeDecl(Point) [type=Record, tab=46, block=0, level=0]
│   │   ├── RecordType [type=Record, tab=-1, block=5, level=1]
│   │   │   ├── FieldDecl(x) [type=Integer, tab=45, block=5, level=1]
│   │   │   │   ├── Type(integer) [type=Integer, tab=33, block=5, level=1]
│   ├── VarDecl(a) [type=Integer, tab=47, block=0, level=0]
│   │   ├── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── VarDecl(a) [type=Real, tab=-1, block=0, level=0]
│   │   ├── Type(real) [type=Real, tab=34, block=0, level=0]
│   ├── VarDecl(b) [type=Boolean, tab=48, block=0, level=0]
│   │   ├── Type(boolean) [type=Boolean, tab=36, block=0, level=0]
│   ├── VarDecl(p) [type=Record, tab=49, block=0, level=0]
│   │   ├── Type(Point) [type=Record, tab=46, block=0, level=0]
├── Block [type=Unknown, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(missing) [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── ArrayAccess [type=Unknown, tab=-1, block=0, level=0]
│   │   │   ├── Var(a) [type=Integer, tab=47, block=0, level=0]
│   │   │   ├── Literal(1) [type=Unknown, tab=-1, block=-1, level=0]
│   │   │   ├── Literal(3) [type=Integer, tab=-1, block=0, level=0]
│   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── FieldAccess(z) [type=Unknown, tab=-1, block=0, level=0]
│   │   │   ├── Var(p) [type=Record, tab=49, block=0, level=0]
│   │   │   ├── Literal(4) [type=Integer, tab=-1, block=0, level=0]
│   ├── If [type=Unknown, tab=-1, block=0, level=0]
│   │   ├── Var(a) [type=Integer, tab=47, block=0, level=0]
│   │   ├── Assign [type=Unknown, tab=-1, block=0, level=0]
│   │   │   ├── Var(b) [type=Boolean, tab=48, block=0, level=0]
│   │   │   ├── Var(true) [type=Boolean, tab=38, block=0, level=0]
│   └── EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifiser	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	InvalidLookupAndAccessProgram		Void		0	1	0	0	43 no
45	x	Field	Integer	0	1	1	0	0	no
46	Point	Type	Record[ref=5]	0	1	0	0	44	no
47	a	Variable	Integer	0	1	0	0	46	no
48	b	Variable	Boolean	0	1	0	1	47	no

49	p	Variable	Record[ref=5]0	1	0	2	48	no
----	---	----------	----------------	---	---	---	----	----

Symbol Table - btab:

idx	last	lpar	psze	vsze

0	49	0	0	0
1	40	0	0	0
2	41	0	0	0
3	42	0	0	0
4	43	0	0	0
5	45	0	0	1

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size

5. Test Case 5

input5.txt
<pre> program InvalidTypesAndOperators; var i: integer; r: real; s: string; b: boolean; arr: array [1..3] of integer; function MaybePositive(x: integer): integer; begin if x > 0 then MaybePositive := x; end; begin i := r; b := i + s; arr[s] := 1; while s do begin i := i + 1; end; for s := 1 to 3 do </pre>

```

begin
  i := i + 1;
end;
end.

```

output5.txt

Semantic Analysis: FAILED

Semantic Errors:

- Function 'MaybePositive' does not assign a return value on all control-flow paths
- Type mismatch: cannot assign Real to Integer
- Operator '+' cannot be applied to Integer and String
- Invalid array index: expected Subrange, got String
- Condition in while statement must be Boolean
- Invalid control variable 's' for 'for' loop (must be ordinal type and properly declared)
- Type mismatch: cannot assign Integer to String
- Type mismatch: cannot assign Integer to String

Decorated AST:

```

Program(InvalidTypesAndOperators) [type=Void, tab=44, block=0, level=0]
├── Declarations [type=Unknown, tab=-1, block=0, level=0]
│   ├── VarDecl(i) [type=Integer, tab=45, block=0, level=0]
│   │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│   ├── VarDecl(r) [type=Real, tab=46, block=0, level=0]
│   │   └── Type(real) [type=Real, tab=34, block=0, level=0]
│   ├── VarDecl(s) [type=String, tab=47, block=0, level=0]
│   │   └── Type(string) [type=String, tab=37, block=0, level=0]
│   ├── VarDecl(b) [type=Boolean, tab=48, block=0, level=0]
│   │   └── Type(boolean) [type=Boolean, tab=36, block=0, level=0]
│   └── VarDecl(arr) [type=Array, tab=49, block=0, level=0]
│       ├── ArrayType [type=Array, tab=-1, block=0, level=0]
│       │   ├── RangeType [type=Subrange, tab=-1, block=0, level=0]
│       │   │   ├── Literal(1) [type=Integer, tab=-1, block=0, level=0]
│       │   │   └── Literal(3) [type=Integer, tab=-1, block=0, level=0]
│       │   └── Type(integer) [type=Integer, tab=33, block=0, level=0]
│       └── FunctionDecl(MaybePositive) [type=Integer, tab=50, block=5, level=1]
│           ├── Declarations(parameters) [type=Unknown, tab=-1, block=5, level=1]
│           │   ├── Param(x) [type=Integer, tab=51, block=5, level=1]
│           │   │   └── Type(integer) [type=Integer, tab=33, block=5, level=1]
│           │   └── Block [type=Unknown, tab=-1, block=5, level=1]
│           │       ├── If [type=Unknown, tab=-1, block=5, level=1]
│           │       │   ├── BinOp(>) [type=Boolean, tab=-1, block=5, level=1]
│           │       │   │   ├── Var(x) [type=Integer, tab=51, block=5, level=1]
│           │       │   │   └── Literal(0) [type=Integer, tab=-1, block=5, level=1]
│           │       │   └── Assign [type=Unknown, tab=-1, block=5, level=1]
│           │       │       ├── Var(MaybePositive) [type=Integer, tab=50, block=5, level=1]
│           │       │       └── Var(x) [type=Integer, tab=51, block=5, level=1]
│           │       └── EmptyStatement [type=Unknown, tab=-1, block=5, level=1]
│           └── Type(integer) [type=Integer, tab=33, block=0, level=0]
└── Block [type=Unknown, tab=-1, block=0, level=0]
    ├── Assign [type=Unknown, tab=-1, block=0, level=0]
    │   ├── Var(i) [type=Integer, tab=45, block=0, level=0]
    │   └── Var(r) [type=Real, tab=46, block=0, level=0]

```

```

├─ Assign [type=Unknown, tab=-1, block=0, level=0]
│   └─ Var(b) [type=Boolean, tab=48, block=0, level=0]
│       └─ BinOp(+) [type=Unknown, tab=-1, block=0, level=0]
│           └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│               └─ Var(s) [type=String, tab=47, block=0, level=0]
├─ Assign [type=Unknown, tab=-1, block=0, level=0]
│   └─ ArrayAccess [type=Integer, tab=-1, block=0, level=0]
│       └─ Var(arr) [type=Array, tab=49, block=0, level=0]
│           └─ Var(s) [type=String, tab=47, block=0, level=0]
├─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
├─ While [type=Unknown, tab=-1, block=0, level=0]
│   └─ Var(s) [type=String, tab=47, block=0, level=0]
│       └─ Block [type=Unknown, tab=-1, block=0, level=0]
│           └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│               └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│                   └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│                       └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│                           └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
│                               └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
├─ For(s) [type=Unknown, tab=47, block=0, level=0]
│   └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
│       └─ Literal(3) [type=Integer, tab=-1, block=0, level=0]
│           └─ Block [type=Unknown, tab=-1, block=0, level=0]
│               └─ Assign [type=Unknown, tab=-1, block=0, level=0]
│                   └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│                       └─ BinOp(+) [type=Integer, tab=-1, block=0, level=0]
│                           └─ Var(i) [type=Integer, tab=45, block=0, level=0]
│                               └─ Literal(1) [type=Integer, tab=-1, block=0, level=0]
│                                   └─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]
└─ EmptyStatement [type=Unknown, tab=-1, block=0, level=0]

```

Symbol Table - tab:

idx	identifrier	obj	type	ref	nrm	lev	adr	link	init
33	Integer	Type	Integer	0	1	0	0	0	no
34	Real	Type	Real	0	1	0	0	33	no
35	Char	Type	Char	0	1	0	0	34	no
36	Boolean	Type	Boolean	0	1	0	0	35	no
37	String	Type	String	0	1	0	0	36	no
38	True	Constant	Boolean	0	1	0	1	37	yes
39	False	Constant	Boolean	0	1	0	0	38	yes
40	writeln	Procedure	Void	1	1	0	0	39	no
41	readln	Procedure	Void	2	1	0	0	40	no
42	write	Procedure	Void	3	1	0	0	41	no
43	read	Procedure	Void	4	1	0	0	42	no
44	InvalidTypesAndOperatorsProgram		Void		0	1	0	0	43 no
45	i	Variable	Integer	0	1	0	0	44	no
46	r	Variable	Real	0	1	0	1	45	no
47	s	Variable	String	0	1	0	3	46	no
48	b	Variable	Boolean	0	1	0	4	47	no
49	arr	Variable	Array[ref=1]	0	1	0	5	48	no
50	MaybePositive	Function	Integer	5	1	0	0	49	no
51	x	Parameter	Integer	0	1	1	0	0	no

Symbol Table - btab:

idx	last	lpar	psze	vsze
0	50	0	0	0

1	40	0	0	0			
2	41	0	0	0			
3	42	0	0	0			
4	43	0	0	0			
5	51	0	0	1			

Symbol Table - atab:

idx	xtyp	etyp	eref	low	high	elsz	size
1	Subrange	Integer	0	1	3	1	3

BAB 4

KESIMPULAN DAN SARAN

1. Kesimpulan

Pada Milestone 3 pengembangan kompilator bahasa Arion ini, kelompok kami telah mengimplementasikan *Semantic Analysis* dengan baik. Struktur yang awalnya berupa *Abstract Syntax Tree* (AST) yang dihasilkan oleh *parser* telah berhasil diubah menjadi *Decorated AST* melalui proses *type annotation*. Implementasi *Symbol Table* mampu mengelola *scope resolution* secara akurat, mulai dari *global scope*, *local scope* di dalam fungsi atau prosedur, hingga menangani *user-defined types* yang rumit seperti *record*.

Kemudian, mekanisme *Type Checking* di dalam program kini sudah mampu memvalidasi kompatibilitas operan pada operasi aritmatika, relasional, maupun boolean. *Control Flow Validation* pada struktur kontrol seperti percabangan (*if-else*, *case*) dan perulangan (*for*, *while*) juga sudah dipastikan berjalan sesuai kaidah semantik bahasa Arion. Keseluruhan proses penelusuran AST dieksekusi menggunakan *Visitor Pattern*. Kelompok kami memilih pendekatan ini untuk memisahkan logika *traversing* dari struktur *node* itu sendiri, sehingga *codebase* tetap modular, bersih, dan mudah di-*maintain* untuk tahap selanjutnya.

2. Saran

Selama mengerjakan analisis semantik ini, kami menemukan beberapa tantangan yang bisa dijadikan catatan untuk milestone-milestone berikutnya. Pertama, dari sisi kode, logika untuk menangani *cascading record access* di dalam modul `parseComponentVariable()` masih perlu dioptimasi lebih lanjut. Penelusuran pointer pada objek bertingkat ini harus dirancang lebih *robust* agar terhindar dari risiko *memory leak* atau *segmentation fault* ketika masuk ke fase eksekusi.

Kedua, mengingat aturan penulisan *grammar* pada spesifikasi tugas cukup sering berubah-ubah, sangat disarankan agar tim membuat sistem *testing* otomatis sebelum lanjut ke tahap berikutnya. Sistem *testing* ini bisa menjadi pengaman, sehingga semisal nanti ada perubahan aturan lagi atau ada perbaikan eror (*bug*), tim bisa langsung mengecek dan memastikan bahwa bagian program yang sudah selesai dan berjalan baik tidak tiba-tiba ikut rusak.

LAMPIRAN

Tautan repository Github:

<https://github.com/Rizelbit/SBT-Tubes-IF2224-2026>

Tabel Pembagian Tugas

No.	NIM	Persentase Pembagian Tugas (%)
1.	13524032	25
2.	13524036	25
3.	13524056	25
4.	13524102	25

REFERENSI

“Introduction to Automata Theory, Languages, and Computation 3rd Edition”

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

“Compilers - Principles, Techniques, and Tools (2006)”

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

“Introduction To Automata Theory Languages , and Computation - John Hopcroft.”

https://methodist.edu.in/web/uploads/files/Introduction%20To%20Automata%20Theory_john%20hopcroft.pdf